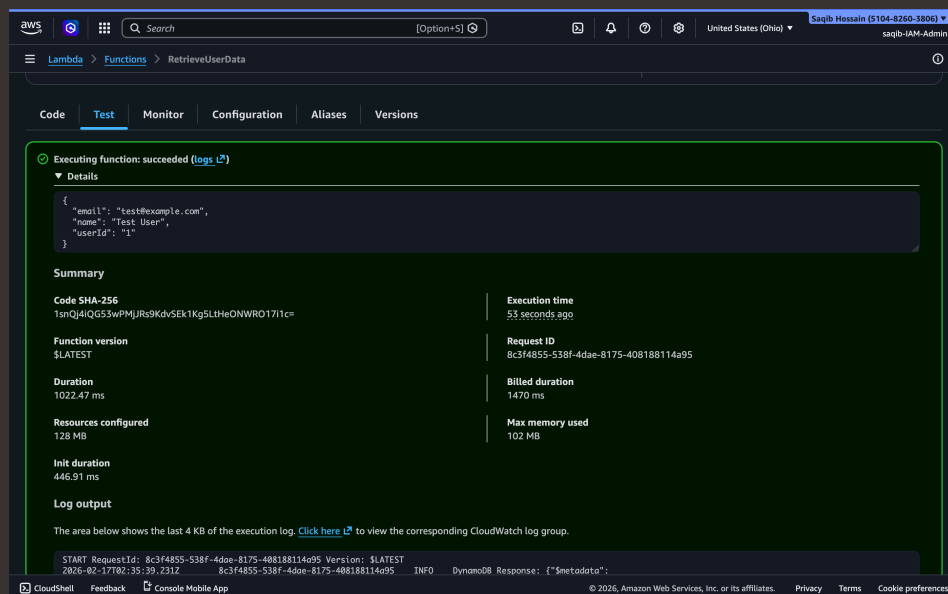




Fetch Data with AWS Lambda



saqibh49@gmail.com





saqibh49@gmail.com

NextWork Student

nextwork.org

Introducing Today's Project!

In this project, I will demonstrate how to pull data from a DynamoDB table using a Lambda function. I'm doing this project to learn how serverless functions work in a web app so I'm better prepared for future projects working on production environments.

Tools and concepts

Services I used were Lambda, DynamoDB and IAM. Key concepts I learnt include Lambda functions, IAM permission policies, and Lambda function testing.

Project reflection

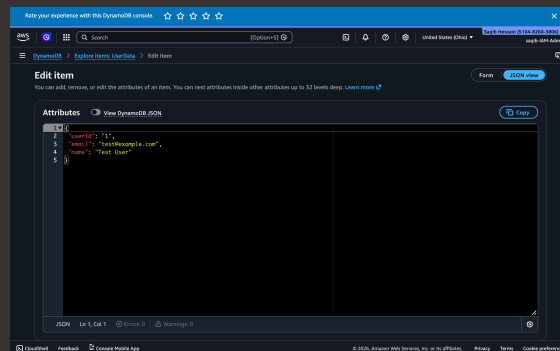
This project took me approximately 1 hour. The most challenging part finding some of the items on the Lambda function page, like the policy permissions, which wasn't super obvious that I needed to click the user's name to get to. It was most rewarding to see the data from my table start populating during my Lambda function's test run.

I chose to do this project today because I'm very interested in cloud engineering and want to know everything I need to know to have a rewarding career in the field someday. Something that would make learning with NextWork even better is if there was a text editor area in the lesson where I could practice my javascript and json syntax.

Project Setup

To set up my project, I created a database using a partition key. The partition key is userID, which means my data will be sorted by each user's ID

In my DynamoDB table, I added an item with some sample user data. DynamoDB is schemaless, which means I don't have to define all my columns upfront like I would with a relational database — I can just add whatever attributes I need to each item, and different items can even have different attributes.



AWS Lambda

AWS Lambda is a service that runs code without needing to manage any computers/servers. Lambda manages them for the user. I'm using Lambda in this project to connect to and retrieve data from my DynamoDB table.



saqibh49@gmail.com

NextWork Student

nextwork.org

AWS Lambda Function

My Lambda function has an execution role, which is a basic IAM role that gives the Lambda function access to only what it needs and nothing more. By default, the role grants access to write logs to CloudWatch.

My Lambda function will take userID as an input and search for data related to it in my DynamoDB table.

The code uses AWS SDK, which is a library of prewritten code that makes it easier for developers to connect to AWS services for their apps. My code uses SDK to communicate with DynamoDB.



saqibh49@gmail.com

NextWork Student

nextwork.org

The screenshot displays the AWS Lambda console interface for the 'RetrieveUserData' function. The 'Code source' tab is active, showing the JavaScript code for the handler. The code imports the 'dynamodb' module and the 'GetCommand' function from the '@aws-sdk/client-dynamodb' package. It initializes a 'dynamodb' client for the 'us-east-2' region. The handler function takes an event object as input, extracts the 'userId' from the event, and constructs a query command to retrieve user data from a DynamoDB table named 'UserData'. The code uses 'dynamodb.send(command)' to execute the query and logs the response. It also includes a catch block to handle any errors that occur during the retrieval process.

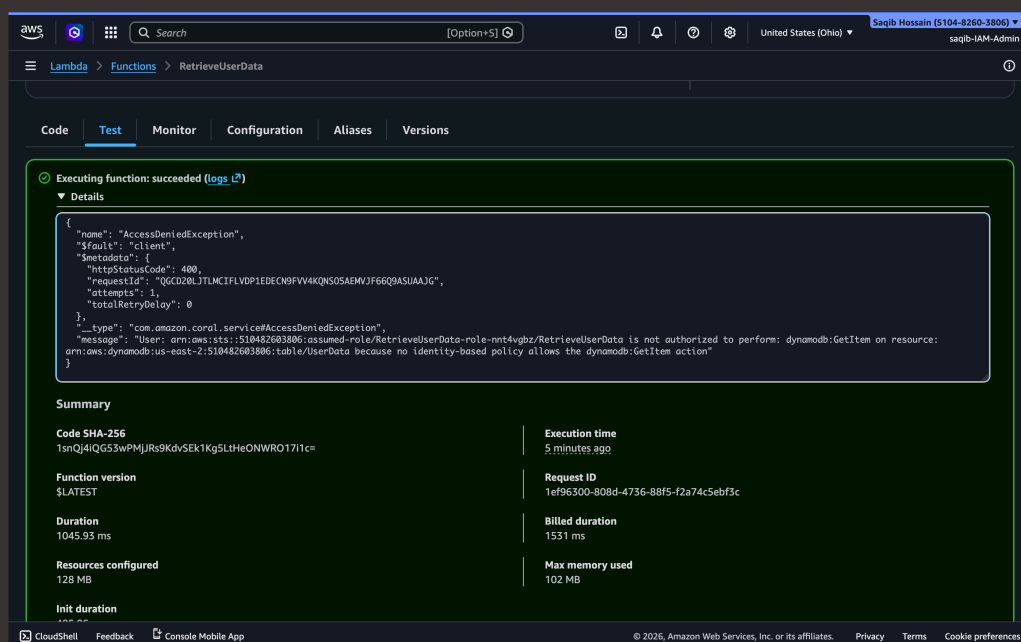
```
1 import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
2 import { DynamoDBDocumentClient, GetCommand } from "@aws-sdk/lib-dynamodb";
3
4 const dbClient = new DynamoDBClient({ region: 'us-east-2' }); // Make sure to replace this with your actual AWS region
5 const db = DynamoDBDocumentClient.from(dbClient);
6
7 async function handler(event) {
8   const userId = String(event.userId); // Make sure to extract userId from the event
9   const params = {
10     TableName: 'UserData',
11     Key: { userId }
12   };
13
14   try {
15     const command = new GetCommand(params);
16     const data = await db.send(command); // Log the raw response from DynamoDB
17     console.log("DynamoDB Response:", JSON.stringify(data));
18     const { Item } = data;
19     if (Item) {
20       console.log("User data retrieved:", Item);
21       return Item;
22     } else {
23       console.log("No user data found for userId:", userId);
24       return null;
25     }
26   } catch (err) {
27     console.error("Unable to retrieve data:", err);
28     return err;
29   }
30 }
31
32 export { handler };
```



Function Testing

To test whether my Lambda function works, I used a command that asks Lambda to search for data related to the `userId 1`. The test is written in JSON. If the test is successful, I'd see the user information I uploaded to my table related to the `userId` of 1.

The test displayed a 'success' because the code ran successfully, indicating no issues in the code, but the function's response was actually denied because we haven't given our Lambda function permission to access DynamoDB yet.



The screenshot shows the AWS Lambda console interface for the `RetrieveUserData` function. The `Test` tab is selected, displaying the following details:

- Executing function:** succeeded (logs [logs](#))
- Details:**

```
{  "name": "AccessDeniedException",  "$fault": "client",  "metadata": {    "httpStatusCode": 400,    "requestId": "06C20LJTLKICFLVDP1EDECN9FV4KQNS0SAEMVJF66Q9ASUAAJG",    "attempts": 1,    "totalRetryDelay": 0  },  "__type": "com.amazon.coral.service#AccessDeniedException",  "message": "User: arn:aws:sts::510482603806:assumed-role/RetrieveUserData-role-mnt4vgbz/RetrieveUserData is not authorized to perform: dynamodb:GetItem on resource: arn:aws:dynamodb:us-east-2:510482603806:table/UserData because no identity-based policy allows the dynamodb:GetItem action"}
```
- Summary:**

Code SHA-256 1snQj4lQG53wPMjJRs9KdvSEk1Kg5LtheONWRO1711c=	Execution time 5 minutes ago
Function version \$LATEST	Request ID 1ef96300-808d-4736-88f5-f2a74c5ebf3c
Duration 1045.93 ms	Billed duration 1531 ms
Resources configured 128 MB	Max memory used 102 MB
Init duration 100.00 ms	

At the bottom of the console, there are links for [CloudShell](#), [Feedback](#), and [Console Mobile App](#). The footer includes the copyright notice: © 2026, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)



saqibh49@gmail.com

NextWork Student

nextwork.org

Function Permissions

To resolve the AccessDenied error I'm adding a DynamoDB read-only policy to it so it can retrieve data from DynamoDB but not be able to edit it.

There were four DynamoDB permission policies I could choose from, but I didn't pick ExecutionRole nor Invocation roles because they only allow for streaming changes to the table, like a news feed and makes Lambda take immediate action on data, respectively. Neither lets me just retrieve the data.

I also didn't pick FullAccess because that gives my function read, write, delete and update access, which is more than it needs. If my function ever became exposed, that would also make the rest of my data insecure as well.

AmazonDynamoDBReadOnlyAccess was the right choice because it provides GET access without allowing editing or deleting capabilities.



saqibh49@gmail.com

NextWork Student

nextwork.org

The screenshot displays the AWS IAM console interface. At the top, a green banner indicates 'Policy was successfully attached to role.' The main content area shows the configuration for the role 'RetrieveUserData-role-nnt4vgbz'. The 'Summary' section includes the creation date (February 16, 2026, 20:47 UTC-05:00), the ARN (arn:aws:iam::510482603806:role/service-role/RetrieveUserData-role-nnt4vgbz), and the maximum session duration (1 hour). The 'Permissions' tab is active, showing two attached policies: 'AmazonDynamoDBReadOnlyAccess' (AWS managed) and 'AWSLambdaBasicExecutionRole-7d5794e...' (Customer managed). The 'Permissions boundary' is currently not set.

RetrieveUserData-role-nnt4vgbz info

Summary

Creation date
February 16, 2026, 20:47 (UTC-05:00)

ARN
arn:aws:iam::510482603806:role/service-role/RetrieveUserData-role-nnt4vgbz

Last activity
22 minutes ago

Maximum session duration
1 hour

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (2) info

You can attach up to 10 managed policies.

Filter by Type: All types

Policy name	Type	Attached entities
AmazonDynamoDBReadOnlyAccess	AWS managed	1
AWSLambdaBasicExecutionRole-7d5794e...	Customer managed	1

Permissions boundary (not set)



saqibh49@gmail.com

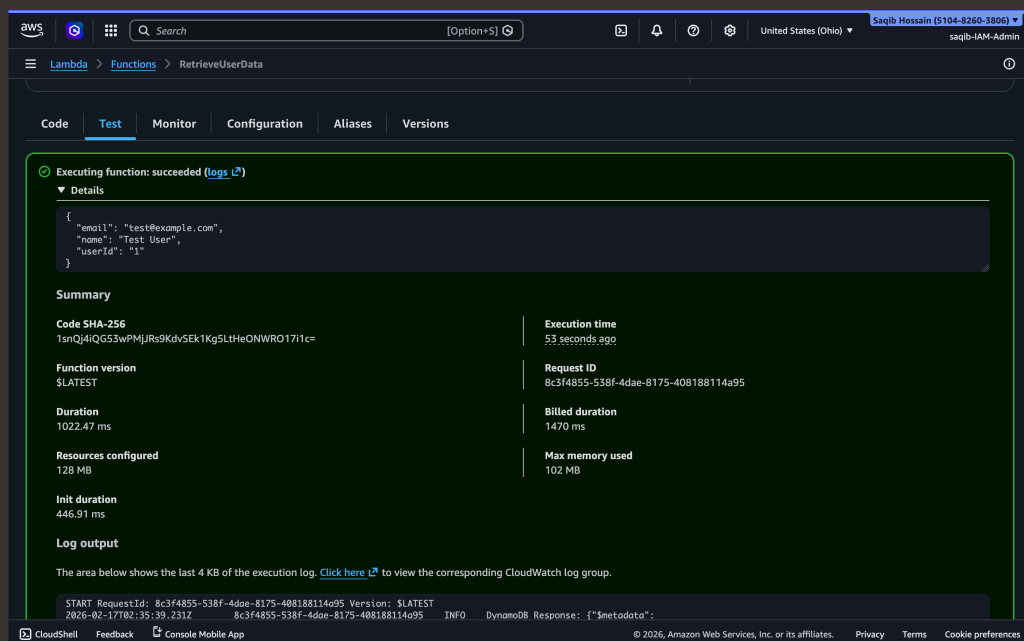
NextWork Student

nextwork.org

Final Testing and Reflection

To validate my new permission settings, I reran my test command. The results were successful because with the ReadOnlyAccess policy attached to my function, it was able to retrieve the relevant data from my table.

Web apps are a popular use case of using Lambda and DynamoDB. For example, I could have Lambda search a DynamoDB table when a user tries to login.



The screenshot displays the AWS Lambda console interface for the 'RetrieveUserData' function. The 'Test' tab is selected, showing a successful execution result. The output JSON is: `{ "email": "test@example.com", "name": "Test User", "userId": "1" }`. The summary section provides details on the execution, including the code SHA-256, function version (\$LATEST), duration (1022.47 ms), and resources configured (128 MB). The log output shows the start of the function execution and the DynamoDB response.

Code	Test	Monitor	Configuration	Aliases	Versions
Executing function: succeeded logs					
Details					
<pre>{ "email": "test@example.com", "name": "Test User", "userId": "1"}</pre>					
Summary					
Code SHA-256 1snQJ4lQGS3wPMJRs9KdvSEk1Kg5LHeONWRO1711c=			Execution time 53 seconds ago		
Function version \$LATEST			Request ID 8c3f4855-538f-4dae-8175-408188114a95		
Duration 1022.47 ms			Billed duration 1470 ms		
Resources configured 128 MB			Max memory used 102 MB		
Init duration 446.91 ms					
Log output					
The area below shows the last 4 KB of the execution log. Click here to view the corresponding CloudWatch log group.					
<pre>START RequestId: 8c3f4855-538f-4dae-8175-408188114a95 Version: \$LATEST 2026-02-17T02:35:39.231Z 8c3f4855-538f-4dae-8175-408188114a95 INFO DynamoDB Response: {"metadata":</pre>					



saqibh49@gmail.com

NextWork Student

nextwork.org

Enhancing Security

For my project extension, I challenged myself to use an inline policy. This will give me more granular control over my function's permissions. In this case, I will use an inline policy to allow my function to only access the UserId table instead of all DynamoDB tables.

To create the permission policy, I used the console because I've used JSON multiple times already so I think it's a good idea to get an idea of the other method of doing things.

When updating a Lambda function's permission policies, you could risk breaking its access to other AWS services if you remove or change the wrong permissions. I validated that my Lambda function still works by running a test that asks it to retrieve data associated with a certain attribute.



saqibh49@gmail.com

NextWork Student

nextwork.org

aws

Search

[Option+S]

Global

Saqib Hossain [5104-8260-3806]

saqib-IAM-Admin

IAM

Roles

RetrieveUserData-role-nnt4vgbz

Create policy

List (6)

Read (Selected 2/29)

All read actions

☒ BatchGetItem

☐ DescribeContinuousBackups

☐ DescribeExport

☐ DescribeImport

☐ DescribeReservedCapacity

☐ DescribeTable

☐ GetAbacStatus

☐ GetResourcePolicy

☐ ListTagsForResource

☐ ReadDataForReplication

☐ ConditionCheckItem

☐ DescribeContributorInsights

☐ DescribeGlobalTable

☐ DescribeKinesisStreamingDestination

☐ DescribeReservedCapacityOfferings

☐ DescribeTableReplicaAutoScaling

☒ GetItem

☐ GetShardIterator

☐ PartiQLSelect

☐ Scan

☐ DescribeBackup

☐ DescribeEndpoints

☐ DescribeGlobalTableSettings

☐ DescribeLimits

☐ DescribeStream

☐ DescribeTimeToLive

☐ GetRecords

☐ ListStreams

☐ Query

Write (38)

Permissions management (3)

Tagging (2)

Resources

Specify resource ARNs for these actions.

All

Specific

table

Specified table resource ARN for the AssociateTableReplica and 53 more actions. Add ARNs to restrict access.

☐ Any in this account

CloudShell

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

